Lab Assignment 6: Creating and Connecting to Databases

DS 6001

Author: Liam Hopkins

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

Please note: you will not be able to use Rivanna for this lab as Rivanna is not set up to work with Docker or with Databases. Problem 1 will guide you through the steps to get databases running on your local system. This can be tricky, so if you have questions or would like some help, please let me know.

Problem 0

Import the following packages:

In [3]:
```
import numpy as np
import pandas as pd
import requests
import json
import sqlite3
import psycopg
import mysql.connector
from sqlalchemy import create_engine
import pymongo
from bson.json_util import loads, dumps
import dotenv
import os
```

Problem 1

Many database systems run as external software on a computer. Some of the software is commercial (Oracle, Microsoft SQL Server), with limited free use and expensive enterprise level licenses. Other database software is entirely free and open source (MySQL, Postgres, SQlite, MongoDB). Regardless of whether the software is free or not, databases are

notoriously difficult to install and start using. More than other kinds of software, database systems seem to run into many errors that are specific to the operating system of your computer. We used to ask students to install Postgres, MySQL, and Mongo, and it was a nightmare because everyone hit some sort of roadblock and those roadblocks were different for every student.

That's just how databases seem to be! Not just for students, but for everyone!

A growing standard practice in industry is to use **containers** to deploy database systems. Please read the [discussion of containers in the course textbook], (https://jkropko.github.io/surfing-the-data-pipeline/docker.html) if you haven't yet done so to familiarize yourself with the purpose and uses of containers and Docker.

This lab will guide you through the installation steps for the software you need to run database systems on your computer (problem 1), build a relational schema (problem 2), document those databases (problem 3), and connect to them through Python with (fingers crossed) as few problems as possible (problem 4 for relational databases, problem 5 for a NoSQL database).

With the exception of SQLite (which operates as a Python package), database systems run as external software that must be installed and run on your computer. To make the installation steps easier, you will need some configuration files that I wrote and saved in a GitHub repository. Open your terminal and then type

```
git clone https://github.com/jkropko/ds6001databases
```

If this command works, it will create a new directory within your current folder called "ds6001databases".

- Check that this folder exists and contains the following files: LICENSE, README.md, compose.yaml, and requirements.txt. (Don't worry if there are a couple extra files.)
- To make things easier, save the notebook file you will be using for your Lab 6 work inside the "ds6001databases" folder

Part a

After cloning the ds6001databases repository, open and examine the file called compose.yaml. Then write answers to the following questions based on your reading of the section on Docker compose files in the textbook:

1. How many containers are being launched when we execute this compose.yaml file?
2. What does each of the containers do?

3. Docker is building these containers from Docker image files that it downloads from a website. On what website are these Docker images stored? (You can provide the general name of this website, no need for the specific URLs)

[4 points]

Part b

If you haven't yet done so, install Docker Desktop on your computer. Go to https://www.docker.com/products/docker-desktop/ and click on the Download button, making sure the operating system listed matches the operating system of your computer (for Apple users, make sure you get the correct version for your computer's processing chip. Click the Apple icon in the top-left corner of your screen, select "About This Mac", and see whether Intel or Apple M1 is listed under Chip).

Once Docker Desktop is installed, find the Docker Desktop program on your computer and run it.

To confirm that Docker Desktop is running, open a terminal and type `docker version`. Copy-and-paste what you see on the screen into your notebook.

If your terminal hangs without displaying anything, that likely means that Docker Desktop is not running or not properly installed. Double check that the Docker Desktop is running. If your terminal is hanging, you can press Control + C to regain access to the command prompt.

[4 points]

Part c

Inside your "ds6001databases" folder, create a file named `.env`. Inside the .env file you need to choose passwords for the MySQL, PostgreSQL, and MongoDB databases, so type

```
MYSQL_ROOT_PASSWORD=password1
POSTGRES_PASSWORD=password2
MONGO_INITDB_ROOT_PASSWORD=password3
MONGO_INITDB_ROOT_USERNAME=mongo
mongo_init_db = mongodb
MYSQL_DATABASE=mysql
```

Change the passwords on the first three lines to whatever you want, but DON'T USE THE @ SYMBOL as that will cause problems. Leave the fourth, fifth, and sixth lines alone, as well as the names of each environmental variable.

Then run this line of Python code:

```
dotenv.load_dotenv()
```

If the .env file has successfully been created and saved in the correct location, the output of this code will be `True`. If `False`, double check the name and location of your file.

[4 points]

Part d

In the terminal, make sure you are in the "ds6001databases" folder (you can check by typing `pwd`. If not, then use `cd` to navigate to the "ds6001databases" folder). Then type

```
docker compose up
```

This command launches all of the containers for running each of the database systems. If successful, you will see a long stream of output with messages that begin `ds6001databases-postgres-1`, `ds6001databases-mysql-1`, and `ds6001databases-mongo-1`. Copy and paste below the first six lines of this output. (Apologies for all the scrolling you might have to do!)

If you receive an error message or if the terminal hangs without displaying output, you will likely need to double-check that parts a, b, and c can still be completed successfully, and that you are working within the "ds6001databases" folder via the terminal.

[4 points]

Part e

To confirm that the databases are running on your system, run the following Python code.

```
# 1. Load needed environment variables
dotenv.load_dotenv()
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD')
MONGO_INITDB_ROOT_USERNAME =
os.getenv('MONGO_INITDB_ROOT_USERNAME')
MONGO_INITDB_ROOT_PASSWORD =
os.getenv('MONGO_INITDB_ROOT_PASSWORD')
MYSQL_ROOT_PASSWORD = os.getenv('MYSQL_ROOT_PASSWORD')

# 2. Test MySQL
dbserver = mysql.connector.connect(
    user='root',
    password=MYSQL_ROOT_PASSWORD,
    host='localhost',
    port='3306',
```

```
    #auth_plugin='mysql_native_password'
)

# 3. Test PostgreSQL
dbserver = psycopg.connect(
    user='postgres',
    password=POSTGRES_PASSWORD,
    host='localhost',
    port = '5432'
)
dbserver.autocommit = True

# 4. Test MongoDB
myclient =
pymongo.MongoClient(f"mongodb://{MONGO_INITDB_ROOT_USERNAME}:
{MONGO_INITDB_ROOT_PASSWORD}@localhost:27017/")
myclient.list_databases()
```

If this code runs without error, you will see output that looks something like

```
<pymongo.synchronous.command_cursor.CommandCursor at 0x177d8b380>
```

and you are all set for working with these databases using Python.

If you do see an error, then you will need to troubleshoot. A few things to try:

- Double check that your work for parts a, b, c, and d can still run successfully.
- Read the error to determine which of the databases is causing the error, to narrow the problem down. Comment-out or delete this part of the code to see if the other databases are working.
- Check the output in the terminal where you issued the `docker compose up` command. Scroll up and down, and look for any lines that write that a container has exited.
- Try a reset of the containers by pressing Control+C in the terminal displaying Docker output, then `docker compose down`, then `docker compose up` again. Then try running the Python code again.
- Sometimes (especially if you've changed the database passwords at some point) incorrect passwords get saved in the Docker volumes and do not change when you make a change to your .env file. To fix this, press Control+C in the terminal displaying Docker output, then `docker compose down`. Then open the Docker Desktop, click on volumes, and find and delete the volumes associated with the "ds6001databases" containers. Then return to the terminal,type `docker compose up`, and try again.

- If all else fails, you are far from the only one to hit a roadblock at this stage. Send me or one of the TAs a message and we can help.

[4 points]

Problem 2

For this problem, I created fake data that represents patients who saw a doctor and received one or more prescriptions. Your goal in this problem is to reorganize the data into a database schema that conforms to E. F. Codd's 3rd normal form, and to document the resulting relational database using an ER diagram.

Wrapping your head around the normal form rules and building and documenting a database schema can be very difficult. First, grant yourself a lot of grace and give yourself plenty of time to take on this problem. Mastering database schema and documentation are central skills that distinguish a data engineer from a data scientist or analyst, so it is worth taking time to think about this task.

The data are listed on the following Google sheet: https://docs.google.com/spreadsheets/d/11wUk5dg39lmI-_O0SQ0M5_5VGgzuGJKvSyMq4Tyxc9M/edit?usp=sharing. For this problem you will be manipulating the data. I only included 10 rows to enable you to use any method you like for working with the data, including point-and-click data manipulation on Google sheets or Excel. Or you can use `pandas` if you like. We are emphasizing the product over the process for now.

There are two tabs on this spreadsheet. The first tab contains the data and the second tab contains column descriptions and notes. These notes (some of which are unrealistic, but needed for this exercise) are:

- No two patients have the same name and date of birth, so `patient_name` and `date_of_birth` together uniquely identify a patient.
- No two drugs share the same brand name, so `prescribed_drug` uniquely identifies the drug.
- The data contain one row for every patient/prescription combination, and no patient receives more than one prescription for the same drug. So `patient_name`, `date_of_birth`, and `prescribed_drug` comprise a primary key in the original data.
- No two physicians in the data share the same name, so `prescribing_physician` uniquely identifies physicians.
- Patients do not change their insurance provider.

- Every physician works at exactly one hospital, no two hospitals share the same name, and every hospital exists at exactly one location.

Some things to keep in mind for this problem:

1. Remember that the first rule for 2nd normal form is that the data are in 1st normal form, and the first rule for 3rd normal form is that the data are already in 2nd normal form. So please focus on meeting the rules for 1st, then 2nd, then 3rd normal form *in that order*.

2. Some students have never worked with databases before, but some have. If you have seen databases before, you have some ideas about how database schema tend to look. But if you arrange the data to match an idea of how databases often look, you will more than likely over-engineer the database by creating too many tables or unnecessary columns. Instead, focus on the explicit rules for each normal form and stop once those rules are met.

In [4]:
```
prescriptions = pd.read_csv("Patient prescriptions - Data.csv")
prescriptions.head()
```

Out[4]:

	patient_name	date_of_birth	prescribed_drug	prior_conditions	patient_sex	patient_insu
0	Nkemdilim Arendonk	2/21/1962	Amoxil	[Pneumonia, Diabetes, Rubella]	M	
1	Nkemdilim Arendonk	2/21/1962	Micronase	[Pneumonia, Diabetes, Rubella]	M	
2	Raniero Coumans	8/15/1990	Zosyn	[Appendicitis, Crohn's disease, Mononucleosis]	M	
3	Raniero Coumans	8/15/1990	Humira	[Appendicitis, Crohn's disease, Mononucleosis]	M	
4	Mizuki Debenham	3/12/1977	Inlyta	[Kidney Cancer]	F	Kaiser Perma

Part a

In words, address the following points regarding 1st normal form:

- Describe the problem or problems that exist in the original data that prevent it from meeting the requirements for 1st normal form.

- Describe a reorganization of the data that does conform to 1st normal form. If this schema has multiple tables, give each table a name that indicates what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

Answer:

- This dataset has one clear problem in it: the column `prior_conditions`. This column contains non-atomic data, as each value in the column is a list of values rather than just a single fact. The problem is larger than that, however, as if we split up this data into seperate columns, these columns would have arbitrary ordering language (i.e. `prior_condition1`, `prior_condition2`, etc.). Also, since not all patients have the same number of primary conditions, not all patients would have a value for each new column, such as a patient with only 2 prior conditions not having a value for `prior_condition5`, for example.

- To reorganize the data, we will split it up into two tables: `prescriptions` and `conditions`. `prescriptions` is our original table with the `prior_conditions` column removed. It will include all the other columns (`patient_name`, `data_of_birth`, `prescribed_drug`, `patient_sex`, `patient_insurance`, `drug_maker`, `drug_cost`, `prescribing_physician`, `physician_medschool`, `physician_years_experience`, `hospital`, and `hospital_location`) and will contain the same number of rows as before. The primary key is the combination of `patient_name`, `date_of_birth`, and `prescribed_drug`. We will then fix the non-atomic data issue with our second table, `conditions`, which contains only the values of prior conditions. It will contain three columns: `patient_name` and `date_of_birth` to identify the unique patient, and `prior_conditions` containing a prior condition corresponding to that patient. For each of a patient's prior conditions, there will be a row in this table with their name in the `patient_name` column, their birthdate in the `date_of_birth` column, and that specific prior condition in the `prior_condition` column. For example, look at the patient Raniero Coumans. To store Raniero's three primary conditions, there would be three rows in the table, containing the values (Raniero Coumans, Appendicitis), (Raniero Coumans, Crohn's disease), and (Raniero Coumans, Mononucleosis). This table will contain more rows than the original data. This is because, as previously stated, this table does not have a row for each patient/prescription combination like before, but rather each patient/prior condition combinations. There are more of these kinds of combinations, so therefore this table will have more rows.

In [5]:
```
conditions = prescriptions[['patient_name', 'date_of_birth', 'prior_conditions']].d
conditions["prior_conditions"] = conditions["prior_conditions"].str.strip('[]').str
conditions = conditions.explode('prior_conditions').reset_index(drop=True)
conditions
```

Out[5]:

	patient_name	date_of_birth	prior_conditions
0	Nkemdilim Arendonk	2/21/1962	Pneumonia
1	Nkemdilim Arendonk	2/21/1962	Diabetes
2	Nkemdilim Arendonk	2/21/1962	Rubella
3	Raniero Coumans	8/15/1990	Appendicitis
4	Raniero Coumans	8/15/1990	Crohn's disease
5	Raniero Coumans	8/15/1990	Mononucleosis
6	Mizuki Debenham	3/12/1977	Kidney Cancer
7	Zoë De Witt	11/23/1947	Cardiomyopathy
8	Zoë De Witt	11/23/1947	Diabetes
9	Zoë De Witt	11/23/1947	Sciatica
10	Zoë De Witt	11/23/1947	Shingles
11	Bonnie Hooper	7/4/1951	Pancreatic Cancer
12	Bonnie Hooper	7/4/1951	Sciatica

In [6]:
```
prescriptions = prescriptions.drop(['prior_conditions'], axis=1)
prescriptions
```

Out[6]:

	patient_name	date_of_birth	prescribed_drug	patient_sex	patient_insurance	drug_m
0	Nkemdilim Arendonk	2/21/1962	Amoxil	M	Aetna	USAntibi
1	Nkemdilim Arendonk	2/21/1962	Micronase	M	Aetna	P
2	Raniero Coumans	8/15/1990	Zosyn	M	Cigna	Ba Internati
3	Raniero Coumans	8/15/1990	Humira	M	Cigna	Ab
4	Mizuki Debenham	3/12/1977	Inlyta	F	Kaiser Permanente	P
5	Zoë De Witt	11/23/1947	Atenolol	F	Medicare	M Pharmaceut
6	Zoë De Witt	11/23/1947	Micronase	F	Medicare	P
7	Zoë De Witt	11/23/1947	Demerol	F	Medicare	P
8	Bonnie Hooper	7/4/1951	Xeloda	F	Blue Cross Blue Shield	Genen
9	Bonnie Hooper	7/4/1951	Demerol	F	Blue Cross Blue Shield	P

Part b

In words, address the following points regarding 2nd normal form:

- Describe the problem or problems that exist in the 1st normal form data that prevent it from meeting the requirements for 2nd normal form.
- Describe a reorganization of the data that does conform to 2nd normal form. If this schema has multiple tables, give each table a name that connects to what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

Answer:

- The problem that exists in the 1st normal form data preventing it from being 2nd normal form is that there are multiple columns that are functionally dependent on part of the primary key, but not all of the primary key. First, the columns with information about the prescribed drug, `drug_maker` and `drug_cost`, are functionally dependent on `prescribed_drug`, but not on `patient_name` or `date_of_birth`. Multiple patients were prescribed multiple drugs, so there is not one drug maker or drug cost matched with a single patient. On the flip side, `patient insurance` is functionally dependent on `patient_name` and `date_of_birth` but not on `prescribed_drug`. Every patient has one insurance, but since multiple different patients were prescribed the same drug some drugs have more than one insurance associated to it.
- My solution for this would be to create two new tables that we will name `drugs` and `medical_info`. The `drugs` table will contain information about the different drugs being prescribed. It will contain the columns `prescribed_drug`, `drug_maker`, and `drug_cost`. The number of rows in this table is lower than in the original, since we have less prescribed drugs than we have patient/prescription combinations. The second table, `insurance`, details the insurance information for a given patient. It has the columns `patient_name`, `date_of_birth`, and `patient_insurance`, and will contain fewer rows than the original since there are fewer patients than there are patient/prescription combinations.

In [7]:
```
drugs = prescriptions[['prescribed_drug', 'drug_maker', 'drug_cost']]
drugs = drugs.drop_duplicates()

prescriptions = prescriptions.drop(['drug_maker', 'drug_cost'], axis=1)
drugs
```

Out[7]:

	prescribed_drug	drug_maker	drug_cost
0	Amoxil	USAntibiotics	14.62
1	Micronase	Pfizer	20.55
2	Zosyn	Baxter International Inc	394.00
3	Humira	Abbvie	7000.00
4	Inlyta	Pfizer	21644.00
5	Atenolol	Mylan Pharmaceuticals	10.58
7	Demerol	Pfizer	37.50
8	Xeloda	Genentech	860.00

In [8]:
```
insurance = prescriptions[['patient_name', 'date_of_birth', 'patient_insurance']]
insurance = insurance.drop_duplicates()
```

```
prescriptions = prescriptions.drop(['patient_insurance'], axis=1)
insurance
```

Out[8]:

	patient_name	date_of_birth	patient_insurance
0	Nkemdilim Arendonk	2/21/1962	Aetna
2	Raniero Coumans	8/15/1990	Cigna
4	Mizuki Debenham	3/12/1977	Kaiser Permanente
5	Zoë De Witt	11/23/1947	Medicare
8	Bonnie Hooper	7/4/1951	Blue Cross Blue Shield

In [9]:
```
prescriptions
```

Out[9]:

	patient_name	date_of_birth	prescribed_drug	patient_sex	prescribing_physician	physici
0	Nkemdilim Arendonk	2/21/1962	Amoxil	M	Earnest Caro	Ca
1	Nkemdilim Arendonk	2/21/1962	Micronase	M	Earnest Caro	Ca
2	Raniero Coumans	8/15/1990	Zosyn	M	Pamela English	Univers
3	Raniero Coumans	8/15/1990	Humira	M	Samantha Bergerson	
4	Mizuki Debenham	3/12/1977	Inlyta	F	Lewis Conti	North
5	Zoë De Witt	11/23/1947	Atenolol	F	Theresa Dahlmans	Lake
6	Zoë De Witt	11/23/1947	Micronase	F	Hilde Ali	Colo
7	Zoë De Witt	11/23/1947	Demerol	F	Hilde Ali	Colo
8	Bonnie Hooper	7/4/1951	Xeloda	F	Steven Garbutt	Ohio S
9	Bonnie Hooper	7/4/1951	Demerol	F	Steven Garbutt	Ohio S

Part c

In words, address the following points regarding 3rd normal form:

- Describe the problem or problems that exist in the 2nd normal form data that prevent it from meeting the requirements for 3rd normal form.
- Describe a reorganization of the data that does conform to 3rd normal form. If this schema has multiple tables, give each table a name that connects to what the rows in the table represent. List the columns that exist in each table, and if the number of rows in a table is different from the number of rows in the original data, explain why.

(It will help you to make these edits to the data now, but you won't show your final data until part d)

[8 points]

Answer:

- The problem that exists in the 2nd normal form data that prevents it from being 3rd normal form is that there are non-primary-key columns that have functional dependence. These columns are the columns for the prescribing physician information: `prescribing_physician`, `physician_medschool`, `physician_years_experience`, `hospital`, and `hospital_location`. All of the information about a specific physician is dependent on `prescribing_physician`.
- My solution for this is to create another new table called `physicians` that stores the information about each physician. It will contain the columns `prescribing_physician`, `physician_medschool`, `physician_years_experience`, `hospital`, and `hospital_location`. The primary key for this table is the `prescribing_physician`. The table contains less rows than the original data since there are fewer physicians than there are patient/prescription combinations.

Important Note: One of the values for `physician_medschool` was missing a space, so it was counting it as a unique physician when it was supposed to be a duplicate. I fixed this issue in Excel. It was for Earnest Caro on row 2 of the original dataset.

In [10]:
```
physicians = prescriptions[['prescribing_physician', 'physician_medschool', 'physic
                            'hospital', 'hospital_location']]
physicians = physicians.drop_duplicates()

prescriptions = prescriptions.drop(['physician_medschool', 'physician_years_experie
                                    'hospital', 'hospital_location'], axis=1)
physicians
```

Out[10]:

	prescribing_physician	physician_medschool	physician_years_experience	hospital	ho
0	Earnest Caro	University of California (Irvine)	14	UPMC Presbyterian Shadyside	
2	Pamela English	University of Michigan	29	Northwestern Memorial Hospital	
3	Samantha Bergerson	Johns Hopkins University	10	Northwestern Memorial Hospital	
4	Lewis Conti	North Carolina State University	8	Houston Methodist Hospital	
5	Theresa Dahlmans	Lake Erie College of Medicine	17	Mount Sinai Hospital	
6	Hilde Ali	Colorado University Boulder	11	Mount Sinai Hospital	
8	Steven Garbutt	Ohio State University	36	UCSF Medical Center	Sa

Part d

Display all of the tables in their entirety from the version of the data you designed for part (c) to meet the requirements of 3rd normal form.

You can use several different methods to display these tables. You can download the tables to CSV format then upload them into your notebook using `pd.read_csv()`, or you can represent them using Markdown table syntax using a tool such as https://www.tablesgenerator.com/markdown_tables, or you can include them as screenshot images if you want. (Please don't just provide external links however, as that will slow down our grading)

[4 points]

Answer: My final tables are listed below:

In [11]:
```
prescriptions
```

Out[11]:

	patient_name	date_of_birth	prescribed_drug	patient_sex	prescribing_physician
0	Nkemdilim Arendonk	2/21/1962	Amoxil	M	Earnest Caro
1	Nkemdilim Arendonk	2/21/1962	Micronase	M	Earnest Caro
2	Raniero Coumans	8/15/1990	Zosyn	M	Pamela English
3	Raniero Coumans	8/15/1990	Humira	M	Samantha Bergerson
4	Mizuki Debenham	3/12/1977	Inlyta	F	Lewis Conti
5	Zoë De Witt	11/23/1947	Atenolol	F	Theresa Dahlmans
6	Zoë De Witt	11/23/1947	Micronase	F	Hilde Ali
7	Zoë De Witt	11/23/1947	Demerol	F	Hilde Ali
8	Bonnie Hooper	7/4/1951	Xeloda	F	Steven Garbutt
9	Bonnie Hooper	7/4/1951	Demerol	F	Steven Garbutt

In [12]:
```
conditions
```

Out[12]:

	patient_name	date_of_birth	prior_conditions
0	Nkemdilim Arendonk	2/21/1962	Pneumonia
1	Nkemdilim Arendonk	2/21/1962	Diabetes
2	Nkemdilim Arendonk	2/21/1962	Rubella
3	Raniero Coumans	8/15/1990	Appendicitis
4	Raniero Coumans	8/15/1990	Crohn's disease
5	Raniero Coumans	8/15/1990	Mononucleosis
6	Mizuki Debenham	3/12/1977	Kidney Cancer
7	Zoë De Witt	11/23/1947	Cardiomyopathy
8	Zoë De Witt	11/23/1947	Diabetes
9	Zoë De Witt	11/23/1947	Sciatica
10	Zoë De Witt	11/23/1947	Shingles
11	Bonnie Hooper	7/4/1951	Pancreatic Cancer
12	Bonnie Hooper	7/4/1951	Sciatica

In [13]:
```
drugs
```

Out[13]:

	prescribed_drug	drug_maker	drug_cost
0	Amoxil	USAntibiotics	14.62
1	Micronase	Pfizer	20.55
2	Zosyn	Baxter International Inc	394.00
3	Humira	Abbvie	7000.00
4	Inlyta	Pfizer	21644.00
5	Atenolol	Mylan Pharmaceuticals	10.58
7	Demerol	Pfizer	37.50
8	Xeloda	Genentech	860.00

In [14]:
```
insurance
```

Out[14]:

	patient_name	date_of_birth	patient_insurance
0	Nkemdilim Arendonk	2/21/1962	Aetna
2	Raniero Coumans	8/15/1990	Cigna
4	Mizuki Debenham	3/12/1977	Kaiser Permanente
5	Zoë De Witt	11/23/1947	Medicare
8	Bonnie Hooper	7/4/1951	Blue Cross Blue Shield

In [15]:
```
physicians
```

Out[15]:

	prescribing_physician	physician_medschool	physician_years_experience	hospital	ho
0	Earnest Caro	University of California (Irvine)	14	UPMC Presbyterian Shadyside	
2	Pamela English	University of Michigan	29	Northwestern Memorial Hospital	
3	Samantha Bergerson	Johns Hopkins University	10	Northwestern Memorial Hospital	
4	Lewis Conti	North Carolina State University	8	Houston Methodist Hospital	
5	Theresa Dahlmans	Lake Erie College of Medicine	17	Mount Sinai Hospital	
6	Hilde Ali	Colorado University Boulder	11	Mount Sinai Hospital	
8	Steven Garbutt	Ohio State University	36	UCSF Medical Center	Sa

Problem 3

For this problem, you will be documenting and a building database that contains the entire collected works of Shakespeare.



The data were collected by Catherine Devlin for https://opensourceshakespeare.org/. The database will have five tables, and they will be brought into your Python environment as `pandas` dataframes in 3rd normal form by running this code:

In [16]:
```
repo = 'https://github.com/jkropko/DS-6001/raw/master/localdata/'
works = pd.read_csv(repo + 'Works.csv')
characters = pd.read_csv(repo + 'Characters.csv')
```

```
chapters = pd.read_csv(repo + 'Chapters.csv')
paragraphs = pd.read_csv(repo + 'Paragraphs.csv')

# convert column names to lowercase (needed for postgreSQL to work properly)
characters.columns = characters.columns.str.lower()
chapters.columns = chapters.columns.str.lower()
paragraphs.columns = paragraphs.columns.str.lower()
works.columns = works.columns.str.lower()

# works in the characters tables is a comma separated list.
# Break it out into multiple rows in a new table
charworks = characters[['charid', 'works']]
charworks.loc['works'] = charworks['works'].str.split(',')
charworks = charworks.explode('works')
charworks = charworks.rename({'works':'workid'}, axis=1)
characters = characters.drop('works', axis=1)

#Remove empty rows
chapters = chapters.query("~chapterid.isnull()")
paragraphs = paragraphs.query("~paragraphid.isnull()")
charworks = charworks.query("~workid.isnull()")

# Add chapterid to paragraphs
paragraphs = pd.merge(paragraphs,
                      chapters.drop('description', axis=1),
                      how='inner',
                      on=['workid', 'section', 'chapter'])

#Remove unnecessary columns
paragraphs = paragraphs.drop(['paragraphtype', 'section', 'chapter'],
                             axis=1)
```

The five tables are:

works: One row per work authored by Shakespeare, with columns:

- `workid` : (primary key) a unique ID without spaces or special characters for the work
- `title` : the title, such as "Twelfth Night"
- `longtitle` : a longer title, if there is one, such as "Twelfth Night, Or What You Will"
- `date` : year of publication
- `genretype` : `t` is a tragedy, such as *Romeo and Juliet* and *Hamlet*; `c` is a comedy, such as *A Midsummer Night's Dream* and *As You Like It*; `h` is a history, such as *Henry V* and *Richard III*; `s` refers to Shakespeare's sonnets; `p` is a narrative (non-sonnet) poem, such as *Venus and Adonis* and *Passionate Pilgrim*
- `notes` : Column for notes from the database maintainer, currently all `NaN`
- `source` : whether the text was originally downloaded from the Moby Project lexicon or Project Gutenberg.
- `totalwords` : Total words in the work
- `totalparagraphs` : Total number of lines of dialogue for plays, or stanzas for poems

characters: One row per character that appears in at least one work by Shakespeare. Some characters, such as Antony or Henry IV, appear in multiple works. Columns:

- `charid` : (primary key) a unique ID for a character
- `charname` : character's name (some characters are different but have the same name, such as the First Musician in Othello and the First Musician in Romeo and Juliet). For poems, the character is "Poet"
- `abbrev` : an abbreviation of the character's name, if needed for reference to some other analyses
- `description` : a longer description of who the character is, if available
- `speechcount` : number of lines of dialogue delivered by the character throughout the works the character appears in

chapters: One row for every unique scene in a play, or for every distinct poem in a collection of poems. Columns:

- `workid` : a unique ID without spaces or special characters for the work
- `chapterid` : (primary key) a unique ID for the scene/poem
- `section` : the scene/poem number
- `chapter` : the act number, if available
- `description` : short description of where the scene takes place, for plays

paragraphs: One row for every line of dialogue that appears in a Shakespeare play, or for every distinct poem in a collection of poems. Columns:

- `workid` : a unique ID without spaces or special characters for the work
- `paragraphid` : (primary key) a unique ID for the line of dialogue/poem
- `paragraphnum` : the position of the paragraph within the ordered list of paragraphs within a chapter
- `charid` : the unique ID of the character delivering the line of dialogue/poem
- `plaintext` : the text of the dialogue/poem
- `phonetictext` : the text of the dialogue/poem in phonetic text, useful for training computers to generate audio of this spoken text
- `stemtext` : the stems of the words in the text, useful for text analyses such as sentiment analysis
- `charcount` : number of characters in the line
- `wordcount` : number of words in the line
- `chapterid` : unique ID for the scene/poem

charworks: One row for every unique combination of character and play. Most characters appear once, but some (such as Antony or Henry IV) appear multiple times. Columns

- `charid` : (primary key) unique ID for the character
- `workid` : (primary key) unique ID for the work

Part a

Please refer to the textbook's discussion of ER diagrams as you complete this problem.

For each of the following pairs of tables, write

1. Which column or columns the tables should be joined on
2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other
3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables
4. Write the database markup language (DBML) code to represent this kind of a relationship.

If we are connecting two tables named `table_a` and `table_b` on a column named `joincolumn` in each table, then the DBML syntax is

- `Ref: table_a.joincolumn - table_b.joincolumn` if the relationship from `table_a` to `table_b` is one-to-one
- `Ref: table_a.joincolumn < table_b.joincolumn` if the relationship from `table_a` to `table_b` is one-to-many
- `Ref: table_a.joincolumn > table_b.joincolumn` if the relationship from `table_a` to `table_b` is many-to-one
- `Ref: table_a.joincolumn <> table_b.joincolumn` if the relationship from `table_a` to `table_b` is many-to-many

If there are multiple columns that must be joined on, provide the DBML code for each of these columns.

Part a, problem i

charworks and **works** [2 points]

In [18]:
```
charworks
```

Out[18]:

	charid	workid
0	1apparition-mac	macbeth
1	1citizen	romeojuliet
2	1conspirator	coriolanus
3	1gentleman-oth	othello
4	1goth	titus
...	...	...
1262	york	henry5
1263	YoungClifford	henry6p2
1264	YoungCoriolanus	coriolanus
1265	YoungLucius	titus
1266	youngsiward	macbeth

1266 rows × 2 columns

In [19]:
```
works
```

Out[19]:

	workid	title	longtitle	date	genretype	notes	source	totalwo
0	12night	Twelfth Night	Twelfth Night, Or What You Will	1599	c	NaN	Moby	198
1	allswell	All's Well That Ends Well	All's Well That Ends Well	1602	c	NaN	Moby	229
2	antonycleo	Antony and Cleopatra	Antony and Cleopatra	1606	t	NaN	Moby	249
3	asyoulikeit	As You Like It	As You Like It	1599	c	NaN	Gutenberg	216
4	comedyerrors	Comedy of Errors	The Comedy of Errors	1589	c	NaN	Moby	146
5	coriolanus	Coriolanus	Coriolanus	1607	t	NaN	Moby	275
6	cymbeline	Cymbeline	Cymbeline, King of Britain	1609	h	NaN	Moby	275
7	hamlet	Hamlet	Tragedy of Hamlet, Prince of Denmark, The	1600	t	NaN	Gutenberg	305
8	henry4p1	Henry IV, Part I	History of Henry IV, Part I	1597	h	NaN	Moby	245
9	henry4p2	Henry IV, Part II	History of Henry IV, Part II	1597	h	NaN	Gutenberg	256
10	henry5	Henry V	History of Henry V	1598	h	NaN	Moby	261
11	henry6p1	Henry VI, Part I	History of Henry VI, Part I	1591	h	NaN	Moby	215
12	henry6p2	Henry VI, Part II	History of Henry VI, Part II	1590	h	NaN	Moby	254
13	henry6p3	Henry VI, Part III	History of Henry VI, Part III	1590	h	NaN	Moby	242
14	henry8	Henry VIII	History of Henry VIII	1612	h	NaN	Moby	246

	workid	title	longtitle	date	genretype	notes	source	totalwo
15	juliuscaesar	Julius Caesar	The Tragedy of Julius Caesar	1599	t	NaN	Moby	197
16	kingjohn	King John	History of King John	1596	h	NaN	Moby	207
17	kinglear	King Lear	The Tragedy of King Lear	1605	t	NaN	Gutenberg	261
18	loverscomplaint	Lover's Complaint	A Lover's Complaint	1609	p	NaN	Moby	25
19	loveslabours	Love's Labour's Lost	Love's Labour's Lost	1594	c	NaN	Moby	214
20	macbeth	Macbeth	The Tragedy of Macbeth	1605	t	NaN	Moby	171
21	measure	Measure for Measure	Measure for Measure	1604	c	NaN	Moby	217
22	merchantvenice	Merchant of Venice	Merchant of Venice, The	1596	c	NaN	Moby	212
23	merrywives	Merry Wives of Windsor	Merry Wives of Windsor, The	1600	c	NaN	Moby	218
24	midsummer	Midsummer Night's Dream	A Midsummer Night's Dream	1595	c	NaN	Moby	165
25	muchado	Much Ado about Nothing	Much Ado about Nothing	1598	c	NaN	Moby	211
26	othello	Othello	The Tragedy of Othello, Moor of Venice	1604	t	NaN	Moby	264
27	passionatepilgrim	Passionate Pilgrim	The Passionate Pilgrim	1598	p	NaN	Moby	31

	workid	title	longtitle	date	genretype	notes	source	totalwo
28	pericles	Pericles	Pericles, Prince of Tyre	1608	h	NaN	Moby	185
29	phoenixturtle	Phoenix and the Turtle	The Phoenix and the Turtle	1601	p	NaN	Moby	3
30	rapelucrece	Rape of Lucrece	The Rape of Lucrece	1594	p	NaN	Moby	150
31	richard2	Richard II	History of Richard II	1595	h	NaN	Moby	224
32	richard3	Richard III	History of Richard III	1592	h	NaN	Moby	292
33	romeojuliet	Romeo and Juliet	The Tragedy of Romeo and Juliet	1594	t	NaN	Moby	245
34	sonnets	Sonnets	Sonnets	1609	s	NaN	Moby	175
35	tamingshrew	Taming of the Shrew	The Taming of the Shrew	1593	c	NaN	Gutenberg	210
36	tempest	Tempest	The Tempest	1611	c	NaN	Moby	166
37	timonathens	Timon of Athens	The Tragedy of Timon of Athens	1607	t	NaN	Moby	182
38	titus	Titus Andronicus	Titus Andronicus	1593	t	NaN	Moby	207
39	troilus	Troilus and Cressida	Troilus and Cressida	1601	t	NaN	Moby	260
40	twogents	Two Gentlemen of Verona	Two Gentlemen of Verona	1594	c	NaN	Moby	171
41	venusadonis	Venus and Adonis	Venus and Adonis	1593	p	NaN	Moby	98
42	winterstale	The Winter's Tale	The Winter's Tale	1610	c	NaN	Moby	249

Answers:

1. The tables should be joined on the workid column. The works table matches to many rows in the charworks table because one work includes many different characters, so one workid will appear multiple times in the charworks workid column.
2. The relationship should be charworks has a many to one relatipship with works table on the work ID column.
3. Ref: charworks.workid > works.workid

Part a, problem ii

characters and **charworks** [2 points]

For each of the following pairs of tables, write

1. Which column or columns the tables should be joined on

The column that should be joined is on charid

2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other

The charid column in the characters table matches to multiple rows in the charid column of charworks table. This is because some characters appear in more than one play. Each character only has one row in characters, but they could have multiple character/play combinations that appear in charworks.

3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables

The tables have a one-to-many relationship on the column charid.

4. Write the database markup language (DBML) code to represent this kind of a relationship.

Ref: characters.charid < charworks.charid

Part a, problem iii

works and **paragraphs** [2 points]

In [21]:
```
works
```

Out[21]:

	workid	title	longtitle	date	genretype	notes	source	totalwo
0	12night	Twelfth Night	Twelfth Night, Or What You Will	1599	c	NaN	Moby	198
1	allswell	All's Well That Ends Well	All's Well That Ends Well	1602	c	NaN	Moby	229
2	antonycleo	Antony and Cleopatra	Antony and Cleopatra	1606	t	NaN	Moby	249
3	asyoulikeit	As You Like It	As You Like It	1599	c	NaN	Gutenberg	216
4	comedyerrors	Comedy of Errors	The Comedy of Errors	1589	c	NaN	Moby	146
5	coriolanus	Coriolanus	Coriolanus	1607	t	NaN	Moby	275
6	cymbeline	Cymbeline	Cymbeline, King of Britain	1609	h	NaN	Moby	275
7	hamlet	Hamlet	Tragedy of Hamlet, Prince of Denmark, The	1600	t	NaN	Gutenberg	305
8	henry4p1	Henry IV, Part I	History of Henry IV, Part I	1597	h	NaN	Moby	245
9	henry4p2	Henry IV, Part II	History of Henry IV, Part II	1597	h	NaN	Gutenberg	256
10	henry5	Henry V	History of Henry V	1598	h	NaN	Moby	261
11	henry6p1	Henry VI, Part I	History of Henry VI, Part I	1591	h	NaN	Moby	215
12	henry6p2	Henry VI, Part II	History of Henry VI, Part II	1590	h	NaN	Moby	254
13	henry6p3	Henry VI, Part III	History of Henry VI, Part III	1590	h	NaN	Moby	242
14	henry8	Henry VIII	History of Henry VIII	1612	h	NaN	Moby	246

	workid	title	longtitle	date	genretype	notes	source	totalwo
15	juliuscaesar	Julius Caesar	The Tragedy of Julius Caesar	1599	t	NaN	Moby	197
16	kingjohn	King John	History of King John	1596	h	NaN	Moby	207
17	kinglear	King Lear	The Tragedy of King Lear	1605	t	NaN	Gutenberg	261
18	loverscomplaint	Lover's Complaint	A Lover's Complaint	1609	p	NaN	Moby	25
19	loveslabours	Love's Labour's Lost	Love's Labour's Lost	1594	c	NaN	Moby	214
20	macbeth	Macbeth	The Tragedy of Macbeth	1605	t	NaN	Moby	171
21	measure	Measure for Measure	Measure for Measure	1604	c	NaN	Moby	217
22	merchantvenice	Merchant of Venice	Merchant of Venice, The	1596	c	NaN	Moby	212
23	merrywives	Merry Wives of Windsor	Merry Wives of Windsor, The	1600	c	NaN	Moby	218
24	midsummer	Midsummer Night's Dream	A Midsummer Night's Dream	1595	c	NaN	Moby	165
25	muchado	Much Ado about Nothing	Much Ado about Nothing	1598	c	NaN	Moby	211
26	othello	Othello	The Tragedy of Othello, Moor of Venice	1604	t	NaN	Moby	264
27	passionatepilgrim	Passionate Pilgrim	The Passionate Pilgrim	1598	p	NaN	Moby	31

	workid	title	longtitle	date	genretype	notes	source	totalwo
28	pericles	Pericles	Pericles, Prince of Tyre	1608	h	NaN	Moby	185
29	phoenixturtle	Phoenix and the Turtle	The Phoenix and the Turtle	1601	p	NaN	Moby	3
30	rapelucrece	Rape of Lucrece	The Rape of Lucrece	1594	p	NaN	Moby	150
31	richard2	Richard II	History of Richard II	1595	h	NaN	Moby	224
32	richard3	Richard III	History of Richard III	1592	h	NaN	Moby	292
33	romeojuliet	Romeo and Juliet	The Tragedy of Romeo and Juliet	1594	t	NaN	Moby	245
34	sonnets	Sonnets	Sonnets	1609	s	NaN	Moby	175
35	tamingshrew	Taming of the Shrew	The Taming of the Shrew	1593	c	NaN	Gutenberg	210
36	tempest	Tempest	The Tempest	1611	c	NaN	Moby	166
37	timonathens	Timon of Athens	The Tragedy of Timon of Athens	1607	t	NaN	Moby	182
38	titus	Titus Andronicus	Titus Andronicus	1593	t	NaN	Moby	207
39	troilus	Troilus and Cressida	Troilus and Cressida	1601	t	NaN	Moby	260
40	twogents	Two Gentlemen of Verona	Two Gentlemen of Verona	1594	c	NaN	Moby	171
41	venusadonis	Venus and Adonis	Venus and Adonis	1593	p	NaN	Moby	98
42	winterstale	The Winter's Tale	The Winter's Tale	1610	c	NaN	Moby	249

In [20]:
```
paragraphs
```

Out[20]:

	workid	paragraphid	paragraphnum	charid	plaintext	phonetictext
0	12night	630863	3	xxx	[Enter DUKE ORSINO, CURIO, and other Lords; Mu...	ENTR TK ORSN KR ANT O0R LRTS MSXNS ATNTNK
1	12night	630864	4	ORSINO	If music be the food of love, play on;\n[p]Giv...	IF MSK B 0 FT OF LF PL ON JF M EKSSS OF IT 0T ...
2	12night	630865	19	CURIO	Will you go hunt, my lord?\n	WL Y K HNT M LRT
3	12night	630866	20	ORSINO	What, Curio?\n	HT KR
4	12night	630867	21	CURIO	The hart.\n	0 HRT
...	...	...	...	...	...	...
35458	winterstale	666323	3460	Paulina	That she is living,\n[p]Were it but told you, ...	0T X IS LFNK WR IT BT TLT Y XLT B HTT AT LK AN...
35459	winterstale	666324	3467	Hermione	You gods, look down\n[p]And from your sacred v...	Y KTS LK TN ANT FRM YR SKRT FLS PR YR KRSS UPN...
35460	winterstale	666325	3475	Paulina	There's time enough for that;\n[p]Lest they de...	0RS TM ENF FR 0T LST 0 TSR UPN 0S PX T TRBL YR...
35461	winterstale	666326	3483	Leontes	O, peace, Paulina!\n[p]Thou shouldst a husband...	O PS PLN 0 XLTST A HSBNT TK B M KNSNT AS I B 0...

	workid	paragraphid	paragraphnum	charid	plaintext	phonetictext
35462	winterstale	666327	3504	xxx	[Exeunt]	EKSNT

35463 rows × 10 columns

1. Which column or columns the tables should be joined on

We would join on the workid column.

2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other The relationship between the works table is one to many with the paragraphgs table on the work id column since one work has multiple paragrapghs it is referencing to/contains.
3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables

The works and paragraphs tables have a one-to-many relationship on the workid column.

4. Write the database markup language (DBML) code to represent this kind of a relationship.

Ref: works.workid < paragraphs.workid

Part a, problem iv

chapters and **works** [2 points]

1. Which column or columns the tables should be joined on

We would join on the workid column.

2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other The relationship between the chapters table is many to one with the waokrs table on the work id column since many chapters reference/contained within on work.
3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables

The chapters and works tables have a many-to-one relationship on the workid column.

4. Write the database markup language (DBML) code to represent this kind of a relationship.

Ref: chapters.workid > works.workid

Part a, problem v

paragraphs and **chapters** [2 points]

1. Which column or columns the tables should be joined on

We would join on the workid column and the chapterid column.

2. A description of your reasoning, in plain language, for whether each table matches to one or many rows in the other The relationship between the chapters table is many to one with the waokrs table on the work id column since many chapters reference/contained within on work.
3. Whether there is a one-to-one, one-to-many, many-to-one, or many-to-many relationship between the tables

The chapters and works tables have a many-to-many relationship on the workid, column.

4. Write the database markup language (DBML) code to represent this kind of a relationship.

Ref: chapters.workid > works.workid

Part a, problem vi

characters and **paragraphs** (count groups that might say a line in unison, like Chorus or Witches, as one character) [2 points]

Part b

Please find the "Using dbdiagram.io and dbdocs.io to Document Your Database" page on Canvas. It is available under Modules and "Getting Started with the Tools We'll Be Using". Read the page and watch the video about using dbdocs.io and dbdiagram.io to document a relational database with an ER diagram.

Use https://dbdiagram.io to create an ER diagram for the five Shakespeare data tables, then publish this diagram to a stable URL by pressing the "Publish to dbdocs" button. Paste a link here to your ER diagram on DBdocs.io. Then paste your DBML code from dbdiagrams.io into the box below.

A few notes:

You can use the `pandas_df_to_dbml()` function, defined below, to generate database markup language (DBML) code for each table that also lists each column's data type, then paste this code into dbdiagram.io (use `print()` around the output to see the correct formatting):

In [4]:
```python
def pandas_df_to_dbml(df: pd.DataFrame, table_name: str) -> str:
    """
    Converts a pandas DataFrame to a DBML string.

    Args:
        df: The pandas DataFrame to convert.
        table_name: The name of the table in the DBML schema.

    Returns:
        A DBML string representing the DataFrame schema.
    """

    dbml_string = f"Table {table_name} {{\n"

    for column_name, column_type in df.dtypes.items():
        dbml_type = map_pandas_dtype_to_dbml_type(column_type)
        dbml_string += f"  {column_name} {dbml_type}\n"

    dbml_string += "}\n"
    return dbml_string

def map_pandas_dtype_to_dbml_type(dtype) -> str:
    """Maps a pandas dtype to a DBML type."""
    dtype_name = str(dtype)
    if "int" in dtype_name:
      return "int"
    if "float" in dtype_name:
      return "float"
    if "datetime" in dtype_name:
        return "datetime"
    return "varchar"
```

Use the syntax [pk] after a column name and data type to designate the columns that are primary keys in each table. In this case the primary keys are

Table	Primary Key column(s)
works	workid
characters	charid
paragraphs	paragraphid
chapters	chapterid
charworks	charid, workid

To draw the lines linking one table to another, include the `Ref:` syntax that you wrote for your answers to part (a).

[4 points]

```
Make sure you get the .ipynb file for this lab from the module 6
page on Canvas

Then when you double-click this box, you'll see three backticks
before and after this text. Leave those alone

Type your code here, between the backticks
```

Problem 4

For this problem, you will use the five dataframes that comprise the Shakespeare database you used in problem 3 to initialize local databases using SQlite, MySQL, and PostgreSQL.

All of the methods for initializing and connecting to databases in Python are listed in the textbook. Note that there are differences in code that depend on whether you are using SQlite, MySQL, or PostgreSQL.

For creating databases on the MySQL and PostgreSQL servers, you will connect directly to the database server and use the `.cursor()` method to interact with it. But once the database exists, please use the `pd.to_sql()` and `pd.read_sql_query()` methods from `pandas` and the `create_engine()` method from `sqlalchemy` (and not the `.cursor()` approach) to add data to or retrieve data from the database.

Before attempting this problem, please make sure your work for problem 1 runs without error, which will ensure your docker containers for MySQL and PostgreSQL are ready to use (SQlite does not require external software and can run without Docker).

Part a

Initialize a new Shakespeare database using SQlite via the `sqlite3` package. Next use the `pd.to_sql()` method to add the five Shakespeare dataframes to this database. Then, to prove that this worked, issue the following SQL query to the database which should display a dataframe listing all characters from Shakespeare's plays with more than 200 lines of dialogue (and the Poet too):

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

[8 points]

In [ ]:

Part b

Use the `dotenv` package to import your MySQL database password into your Python environment (don't expose this password in your code). Then use this password to initialize a new Shakespeare database using MySQL and the `mysql.connector` package.

Next, use the `create_engine()` method from `sqlalchemy` and the `pd.to_sql()` method to add the five Shakespeare dataframes to this database. Then, to prove that this worked, use the `pd.read_sql_query()` method to issue the following SQL query to the database

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

[8 points]

Part c

Use the `dotenv` package to import your PostgreSQL database password into your Python environment (don't expose this password in your code). Then use this password to initialize a new Shakespeare database using PostgreSQL and the `psycopg` package. (For PostgreSQL, you will need to set the `.autocommit` attribute of Python variable that connects to the PostgreSQL server to `True`, or you will receive cryptic errors.)

Next, use the `create_engine()` method from `sqlalchemy` and the `pd.to_sql()` method to add the five Shakespeare dataframes to this database (don't worry if a negative number is displayed. This is a known issue with PostgreSQL in Python but is not indicative of a problem). Then, to prove that this worked, use the `pd.read_sql_query()` method to issue the following SQL query to the database

```
SELECT charname, description, speechcount
FROM characters
WHERE speechcount > 200
```

Finally, after running the query, use the `.commit()` and `.close()` methods on the Python variable containing the database connection to save your changes and prevent further changes.

(If you see an error like `ObjectInUse: database "shakespeare" is being accessed by other users DETAIL:  There is 1 other session using the database.`, go to the terminal, type Control+C, then `docker compose down`, then `docker compose up`, then try again.)

[8 points]

Problem 5

Colin Mitchell is a web-developer and artist who has a bunch of cool projects that play with what data can do on the internet. One of his projects is Today in History, which provides an API to access all the Wikipedia pages for historical events that happened on this day in JSON format. The records in this JSON are stored in the `['data']['events']` path. Here's the first listing for today (the day I ran this code):

In [5]:
```
history = requests.get("https://history.muffinlabs.com/date")
history_json = json.loads(history.text)
events = history_json['data']['Events']
events[0]
```

Out[5]:
```
{'year': '4',
 'text': 'Augustus adopts Tiberius.',
 'html': '4 - <a href="https://wikipedia.org/wiki/Augustus" title="Augustus">Augus
tus</a> adopts <a href="https://wikipedia.org/wiki/Tiberius" title="Tiberius">Tibe
rius</a>.',
 'no_year_html': '<a href="https://wikipedia.org/wiki/Augustus" title="Augustus">A
ugustus</a> adopts <a href="https://wikipedia.org/wiki/Tiberius" title="Tiberius">
Tiberius</a>.',
 'links': [{'title': 'Augustus',
   'link': 'https://wikipedia.org/wiki/Augustus'},
  {'title': 'Tiberius', 'link': 'https://wikipedia.org/wiki/Tiberius'}]}
```

Here's the count of total events for the day I ran this code:

In [6]:
```
len(events)
```

Out[6]:
```
70
```

For this problem, you will use MongoDB and the `pymongo` package to create a local document store NoSQL database containing these historical events.

Part a

First, check that your work for problem 1 is still running without error. If so, then you have a MongoDB server running inside a Docker container.

Use `pymongo` to connect to the local MongoDB client, create a database named "history" and a collection within that database named "today". Because you will probably be running

this code several times as you work, debugging as you go along, it's useful write code that deletes any existing "today" collections before creating a new "today" collection.

[4 points]

Part b

Insert all of the records in `events` into this collection. [4 points]

Part c

Issue the following query to find all of the records whose text contain the word "Virginia":

```
query = {
    "text":{
        "$regex": 'England'
    }
}
```

If there are no results that contain the word "England", choose a different word like "China" or "war". Display the count of the number of documents that match this query, display the output of the query, and generate a JSON formatted variable containing the output. [4 points]

Problem 6

Once you are finished working with databases, clear up the space on your computer by going to the terminal that you used to launch the Docker containers, press CONTROL + C on your keyboard to stop the containers, then type `docker compose down` to disconnect the volumes and networks. It's a good idea to make a practice out of doing these steps when you finish working with databases.

This problem isn't graded, and no need to write anything. But please do this anyway.